

Configuración de Servicios: SSH y Enrutamiento

Parte 1: Teoría y Hardening

1. Por qué hacemos un laboratorio de SSH si se instala solo?

A día de hoy, es verdad que en cualquier distribución moderna de Linux (como Debian, Ubuntu Server o RHEL), SSH (OpenSSH Server) viene instalado por defecto o es una simple tilde durante la instalación. El servicio levanta solo y funciona.

El problema es que la configuración que trae de fábrica está pensada para la comodidad, no para la seguridad. Dejar el SSH con los valores por defecto es como dejar la llave de la casa abajo de la alfombra. El puerto 22 (el puerto estándar de SSH) es el vector de ataque más escaneado en todo internet. Si levantas un servidor en la nube pública y dejás el SSH abierto con usuario/contraseña, en menos de 5 minutos vas a tener miles de bots intentando romper tu clave root mediante ataques de fuerza bruta.

Entonces, como buena práctica, no "instalamos" SSH; hacemos Hardening y lo usamos como una herramienta de ingeniería de redes para crear túneles seguros.

2. Introducción Teórica: La Arquitectura de Secure Shell

SSH es un protocolo de red (y un conjunto de herramientas) que opera en la arquitectura Cliente-Servidor. Su propósito es permitirnos administrar equipos remotos de forma cifrada, reemplazando a protocolos prehistóricos e inseguros como Telnet, que mandaban todo el texto (incluyendo contraseñas) en texto plano.

~ **El Cliente:** Es el programa ssh que corremos en nuestra máquina (o programas como PuTTY/MobaXterm en Windows).

~ **El Servidor (Demonio):** Es el proceso sshd que está corriendo en segundo plano en el servidor remoto, escuchando conexiones (por defecto en el puerto TCP 22).

La Criptografía detrás de SSH

Para que la conexión sea segura y nadie en el medio pueda leer lo que escribimos (Man-in-the-Middle), SSH usa dos tipos de cifrado:

1. **Asimétrico (Para el handshake y la identidad):** Usa pares de llaves (Pública y Privada). Sirve para que el servidor pruebe que es quien dice ser, y para que nosotros nos autentiquemos sin mandar contraseñas.
2. **Simétrico (Para el túnel de datos):** Una vez que nos conectamos, el cliente y el servidor negocian una llave temporal única y rapidísima (ej. AES) para cifrar todo el texto que va y viene durante esa sesión.

3. Los Tres Pilares del Hardening en SSH

En la industria actual (bajo normativas Zero Trust), un servidor de producción tiene que cumplir con estas tres reglas en su configuración (sshd_config):

A. Destrucción de la Autenticación por Contraseña

Las contraseñas de texto ya no sirven para infraestructura crítica; se filtran, se roban o se rompen. La norma es usar **Pares de Llaves Criptográficas (Public/Private Keys)**.

Actualización de industria: Históricamente se generaban llaves RSA. Hoy en día, RSA se considera lento y pesado. El estándar corporativo es **Ed25519** (Criptografía de Curva Elíptica), que genera llaves súper cortas, rapidísimas y matemáticamente casi irrompibles.

B. Prohibir el acceso directo de Root (PermitRootLogin no)

Nunca, bajo ningún concepto, se le permite al usuario root loguearse directamente por SSH. El ataque de fuerza bruta más común en internet intenta adivinar contraseñas contra el usuario "root" porque saben que existe en todos los Linux. Al bloquearlo de raíz en el archivo de configuración, mitigamos el 99% de estos ataques. La práctica correcta es entrar con un usuario común (ej: lucca) y luego escalar privilegios con sudo o su -.

C. Seguridad Perimetral y Control de Acceso (AllowUsers y Fail2Ban)

No alcanza con tener llaves fuertes. Tenemos que decirle al daemon qué usuarios específicos tienen permiso para entrar (AllowUsers lucca lucca2). Además, se suele acompañar con una herramienta de capa superior como **Fail2Ban**, que lee los logs en tiempo real y, si ve que una IP se equivoca tres veces seguidas intentando entrar, la bloquea directamente en el Firewall (Nftables/UFW).

4. La Navaja Suiza: Los Túneles SSH (Port Forwarding)

Más allá de tirar comandos remotamente, SSH nos permite encapsular otro tráfico de red dentro de su túnel cifrado. Esto se usa muchísimo en ciberseguridad para puentear firewalls o acceder a servicios internos sin exponerlos a internet.

Ejemplo Práctico (Local Port Forwarding):

Imaginá que tu empresa tiene una base de datos MySQL corriendo en un servidor interno (10.0.0.50 puerto 3306). Esa base de datos no tiene IP pública, y el firewall bloquea todo acceso desde afuera.

Si vos tenés acceso por SSH a un servidor web de la DMZ que sí ve a esa base de datos, podés usar SSH para abrir un túnel. Le decís a tu cliente SSH: "Todo lo que yo mande a mi puerto local 3306, metelo en el túnel cifrado, llevalo hasta el servidor de salto, y desde ahí escupilo hacia la IP 10.0.0.50 en su puerto 3306".

A nivel práctico, es como tirar un cable de red virtual larguísimo y seguro desde tu casa directo a la SQL de la empresa, saltándote las restricciones perimetrales.

Laboratorio Práctico: Hardening de SSH y Port Forwarding.

Antes de empezar, aclaremos la arquitectura que seguiré en este laboratorio:

1. El Servidor Objetivo: En mi caso usare la VM del Servidor DNS que subí anteriormente. Ese será el objetivo a asegurar. Porque un servidor DNS nunca, jamás, debe tener el SSH abierto con contraseñas.
2. El "Servicio Oculto" (Para la prueba del Túnel): Para probar el Port Forwarding, necesitamos un servicio que esté corriendo adentro de esa VM (DNS), pero que solo escuche peticiones locales (localhost) y rechace conexiones desde la red externa. Vamos a usar el puerto 53 del DNS (o instalaremos un servidor web liviano rapidito de prueba) para simular esa "base de datos interna" a la que solo podemos llegar a través del túnel.
3. El Cliente: desde el que vamos a lanzar los ataques y establecer el túnel desde nuestra máquina anfitriona (la máquina física donde corre el hipervisor, KVM en mi caso) hacia la máquina virtual.

IMPORTANTE!! Esta primera fase se ejecuta en tu máquina física (el cliente), NO dentro de la máquina virtual o el servidor. Primero armamos nuestro "documento de identidad" antes de ir a golpear la puerta del servidor.

Parte 1: Creación de la Llave Criptográfica (El "DNI" del Sysadmin)

Abriremos una terminal en nuestro S.0 físico

```
lucky4@Luck4-Debian:~
OS: Debian GNU/Linux 13 (trixie) x86_64
Host: H610M H DDR4 (-CF)
Kernel: Linux 6.12.96+deb13-amd64
Uptime: 15 mins
Packages: 1933 (dpkg), 9 (flatpak), 7 (snap)
Shell: zsh 5.9
Display (LG ULTRAGEAR): 1920x1080 @ 240 Hz in 26" [External] *
Display (GM3CP238): 1920x1080 @ 144 Hz in 24" [External]
DE: GNOME 48.7
WM: Mutter (X11)
WM Theme: Nordic
Theme: Nordic [GTK2/3/4]
Icons: Adwaita [GTK2/3/4]
Font: Cantarell (11pt) [GTK2/3/4]
Cursor: Adwaita (24px)
Terminal: GNOME Terminal 3.56.2
Terminal Font: Hack Nerd Font (11pt)
CPU: 12th Gen Intel(R) Core(TM) i5-12400F (12) @ 4.40 GHz
GPU: NVIDIA GeForce RTX 3090 [Discrete]
Memory: 5.65 GiB / 31.17 GiB (18%)
Swap: 0 B / 7.54 GiB (0%)
Disk (/): 60.05 GiB / 431.15 GiB (14%) - ext4
Local IP (enp3s0): 192.168.1.38/24
Locale: en_US.UTF-8
```

1. Generar el par de llaves asimétricas:

No usaremos el comando `rsa`, quedo deprecado, usaremos la criptografía moderna.

~ `ssh-keygen -t ed25519 -C "usuario@prueba"`

```
> ssh-keygen -t ed25519 -C "lucca@admin-redes"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/luck4/.ssh/id_ed25519):
Enter passphrase for "/home/luck4/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/luck4/.ssh/id_ed25519
Your public key has been saved in /home/luck4/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:zmxISjyeqFmi@aLZ7WSh7PNCzoRSbpxSHCUWbzSRwWs lucca@admin-redes
The key's randomart image is:
+--[ED25519 256]--+
|  ++*+          |
|  ..+0.         |
|   .0.          |
|   +0E          |
|  * 0= . S      |
|+0B= * =        |
|=B* * . =       |
|0@=+ .          |
|* 0=+          |
+----[SHA256]-----+
```

- Esto genera un par matemático de llaves. La Privada (id_ed25519) es tu secreto máximo; nunca debe salir de tu computadora. La Llave Pública (id_ed25519.pub) es el candado que le vamos a mandar al servidor.
- Cuando el comando te pregunte dónde guardarlo: Puedes darle Enter para aceptar la ruta por defecto (~/.ssh/).
- Cuando te pida una "passphrase" (contraseña de llave): Como es un entorno de laboratorio, le damos Enter dos veces para dejarla vacía, también podemos ponerle una contraseña para máxima seguridad (si te roban la computadora, no pueden usar tu llave sin esa contraseña).

2. Enviar el "candado" al servidor (Copia de la llave pública)

Ahora, le vamos a mandar la llave pública a nuestro usuario del servidor

```
~ ssh-copy-id root@"IP servidor"root
```

```

$ ssh-copy-id luck4@192.168.122.20
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: ssh-add -L
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
luck4@192.168.122.20's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'luck4@192.168.122.20'"
and check to make sure that only the key(s) you wanted were added.
```

- Se conecta temporalmente usando tu contraseña, toma el contenido de tu id_ed25519.pub y lo pega dentro del archivo ~/.ssh/authorized_keys del servidor.

Nota: no se recomienda tener el login de root habilitado por defecto. En algunas distros el daemon SSH, por defecto, prohíbe que el usuario root inicie sesión usando contraseña. Es una medida de protección básica para evitar que scripts automatizados adivinen la clave de root desde internet.

Ahora si hacemos un ssh, entramos directo sin que nos pida la passwd por que el handshake funciona.

~ ssh user@"IP servidor"

```
> ssh luck4@192.168.122.20
Linux UTN-Lucka 6.12.95+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.95-1 (2026-07-04) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Jul 20 14:37:21 2026 from 192.168.122.1
luck4@UTN-Lucka:~$
```

Parte 2: Hardening

Ahora vamos a meter mano en el archivo de configuración del daemon SSH y cambiaremos algunos parametros importantes.

~ vim /etc/ssh/sshd_config

Lo que haremos en este archivo sera:

1. Apagar el ingreso del administrador supremo:

En la línea PermitRootLogin. Suele decir prohibit-password o yes. Cambiamos a: **PermitRootLogin no**

2. Destruir la autenticación por contraseñas:

Buscamos PasswordAuthentication. Esta es la línea que frena los ataques de fuerza bruta de los bots.

Cambiamos a: **PasswordAuthentication no**

3. Garantizar la lectura de tu llave:

La línea **PubkeyAuthentication** tiene que estar en **yes** (gral esta así por defecto, pero no está de más revisarlo).

4. Lista Blanca de Usuarios (filtro):

Esta línea generalmente no existe, así que la agregamos a mano al final del archivo. Reemplaza tu_usuario por el usuario con el que copiaste la llave: **AllowUsers tu_usuario**

```

Include /etc/ssh/sshd_config.d/*.conf

#Port 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes

# To disable tunneled clear text passwords, change to "no" here!
PasswordAuthentication no
#PermitEmptyPasswords no

# Change to "yes" to enable keyboard-interactive authentication. Depending on
# the system's configuration, this may involve passwords, challenge-response,
# one-time passwords or some combination of these and other methods.
# Beware issues with some PAM modules and threads.
KbdInteractiveAuthentication no

```

Para que los cambios efectuen reiniciamos con:
~ systemctl restart sshd

Si los cambios fueron aplicados correctamente, el login de root tendría que salir así:

```
> ssh root@192.168.122.20  
root@192.168.122.20: Permission denied (publickey).
```

Parte 3: Enrutamiento (Local Port Forwarding)

El objetivo de esta parte es acceder a un servicio interno que está bloqueado para el exterior (simulando una base de datos o una intranet), utilizando nuestro túnel SSH seguro como puente.

Para probarlo, no vamos a instalar un motor de bases de datos pesado; vamos a usar Python (que ya viene preinstalado en Linux) para levantar un mini servidor web de prueba, pero con una condición: **solo va a escuchar en su propia interfaz de localhost (127.0.0.1).**

1.Preparar el servicio oculto (En el Servidor)

Vamos a levantar un servidor web en el puerto 8080, pero exclusivamente en el loopback de la maquina virtual, con este comando:

```
~ python3 -m http.server 8080 --bind 127.0.0.1
```

```
root@UTN-Lucka:~# python3 -m http.server 8080 --bind 127.0.0.1  
Serving HTTP on 127.0.0.1 port 8080 (http://127.0.0.1:8080/) ...
```

Veremos que la terminal se queda colgada diciendo Serving HTTP on 127.0.0.1 port 8080...

Lo dejaremos asi y desde la maquina anfitriona trataremos de conectarnos a la ip del servidor en ese puerto.
(en mi caso seria `http://192.168.122.20:8080`)

Unable to connect

Firefox can't establish a connection to the server at 192.168.122.20:8080.

- The site could be temporarily unavailable or too busy. Try again in a few moments.
- If you are unable to load any pages, check your computer's network connection.
- If your computer or network is protected by a firewall or proxy, make sure that Firefox is permitted to access the web.

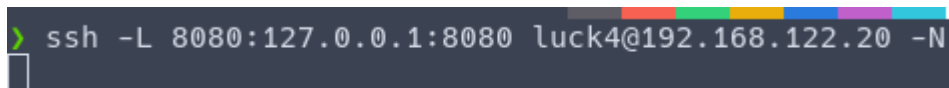
Try Again

Lógicamente la página va a dar error o decir que rechazó la conexión. El servicio existe, pero como está atado a 127.0.0.1 interno del servidor, desde tu red física no podés llegar.

Parte 3: El tunel SSH

Acá está la magia, abriremos una nueva terminal en la máquina física (anfitrión) y usaremos este comando exacto:

```
~ ssh -L 8080:127.0.0.1:8080 tuUsuario@tuIP -N
```



```
> ssh -L 8080:127.0.0.1:8080 luck4@192.168.122.20 -N
```

Como funciona este comando:

-L 8080: Le dice al cliente SSH que abra el puerto 8080 en tu propia máquina física.

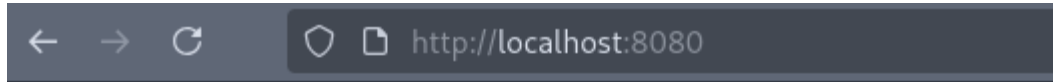
:127.0.0.1:8080 Es el destino final *desde la perspectiva del servidor*. Le dice: "Todo lo que entre por este túnel, tiralo hacia 127.0.0.1:8080 allá adentro".

tu_usuario@tuIP Es la puerta de entrada (el servidor) al que nos conectamos.

-N Le dice al SSH: "Manten el túnel de red abierto, sin abrir una terminal".

Ahora vamos a ir al navegador de nuevo y en lugar de buscar la ip del servidor, buscaremos la propia, <http://localhost:8080>

Si la configuración fue correcta tendria que abrir el directorio del servidor.



Directory listing for /

- [.bash_history](#)
 - [.bashrc](#)
 - [.config/](#)
 - [.profile](#)
 - [.ssh/](#)
 - [.viminfo](#)
-

Lo que acaba de pasar es que tu navegador le pegó al puerto 8080 de tu máquina física. El cliente SSH agarró ese paquete, lo cifró, lo metió por el puerto 22, cruzó la red física, entró al servidor, se descifró, y se lo entregó al servicio interno de Python como si la petición se hubiera originado desde adentro de la propia máquina virtual.

Esto demuestra la capacidad de SSH para saltarse topologías de red complejas dándonos un enrutamiento seguro.